

5. Project Development Phase

ComicCraft – AI Comic Story Creator Using Gemini Models

Implementing the AI generation pipeline, backend, and frontend

1. Development Overview

ComicCraft is developed as a web application using FastAPI, Jinja2 templates, Gemini models, Stable Diffusion, and FPDF. The development work connects the separate generation components into a complete workflow.

2. Environment Setup

Set up and activate a Python virtual environment, install the required project packages, and configure the Gemini API key and Hugging Face token through environment variables. The local application can be started using the command `uvicorn app.main:app --reload`.

3. Outline Generation

The `generate_outline()` function in `gemini_flash.py` uses Gemini Flash to produce a structured five-panel outline. Every panel contains a panel number, title, scene description, and image-generation prompt.

4. Story Generation

The `generate_story()` function in `gemini_pro.py` expands the generated outline into a comic-style story, adding narration and character dialogue for each panel.

5. Image Generation

The `generate_image()` function in `image_generator.py` uses Stable Diffusion to create an illustration from each panel's image prompt. The prompt is sanitized for safe filenames, and the generated image is stored in `static/panels`.

6. Layout Assembly

The `build_comic_layout()` function in `layout_builder.py` combines the generated story and images, pairing every image with its corresponding panel text. The final layout contains panel numbers, image paths, and text.

7. PDF Export

The `save_pdf()` function in `exporters.py` uses FPDF to assemble the comic into a multi-page PDF. Panel images and narration are arranged on consecutive pages, and the exported PDF is stored in `static/exports`.

8. FastAPI Integration

`routes.py` connects the different functions. The `/generate` route handles form submissions, while `/generate-comic/json` handles JSON requests and returns layout information with a PDF path. The `/test-image` route allows image generation to be tested separately. The homepage and export-success routes render their corresponding templates.

9. Frontend Development

The HTML, CSS, and Jinja2 interface provides `index.html` for user input, `comic_preview.html` for panel-by-panel viewing, and `export_success.html` for export confirmation. Jinja2 loops through the layout data to display the panel content dynamically.

10. Development Outcome

The development phase combines input collection, AI-based outline and story generation, illustration creation, layout assembly, preview rendering, and PDF export into a single application workflow.